
Curriculum-Guided Layer Scaling for Language Model Pretraining

Karanpartap Singh¹ Neil Band¹ Ehsan Adeli¹

Abstract

As the cost of pretraining large language models grows, there is continued interest in strategies to improve learning efficiency during this core training stage. Motivated by cognitive development, where humans gradually build knowledge as their brains mature, we propose **Curriculum-Guided Layer Scaling (CGLS)**, a framework for compute-efficient pretraining that synchronizes increasing data difficulty with model growth through progressive layer stacking (i.e., gradually adding layers during training). At the 100M parameter scale, using a curriculum transitioning from synthetic short stories to general web data, CGLS outperforms baseline methods on the question-answering benchmarks PIQA and ARC. Pretraining at the 1.2B scale, we stratify the DataComp-LM corpus with a DistilBERT-based classifier and progress from general text to highly technical or specialized content. Our results show that progressively increasing model depth alongside sample difficulty leads to better generalization and zero-shot performance on various downstream benchmarks. Altogether, our findings demonstrate that CGLS unlocks the potential of progressive stacking, offering a simple yet effective strategy for improving generalization on knowledge-intensive and reasoning tasks.

1. Introduction

Large language models (LLMs) are typically pretrained in a single, continuous pass, processing all tokens with a uniform amount of computation regardless of their complexity or relevance to downstream tasks of interest. While this approach has shown remarkable success in large-scale models like GPT-4 (OpenAI et al., 2023) and Llama 3 (Dubey et al.,

2024), it differs significantly from how humans learn, often leading to models that excel in generating coherent text but struggle with long-context reasoning (Schnabel et al., 2025). Recent works like Phi-3 (Abdin et al., 2024), MiniCPM (Hu et al., 2024), and others (Feng et al., 2024) have explored *midtraining*, adjusting the training data distribution partway through training by incorporating higher-quality, multilingual, or long-form text. However, this coarse-grained curriculum is applied on *fixed* model architectures. Inspired by how humans progressively build knowledge alongside their physically growing brains, we explore whether gradually scaling a model in tandem with increasingly complex data can enable more efficient and effective learning.

Curriculum learning (Bengio et al., 2009) has shown success in guiding models from easier to harder tasks, but remains largely unexplored in modern language model (LM) pretraining. Recent studies propose several alternative methods to schedule complexity and model scale during LM training. Fan & Jaggi (2023) train small proxy models to determine sample learnability, enabling complexity-aware curricula for training LMs. Progressive layer stacking (Gong et al., 2019; J. Reddi et al., 2023; Saunshi et al., 2024; Gu et al., 2020; Yang et al., 2020; Panigrahi et al., 2024), layer-dropping (Zhang & He, 2020), and pruning (Kim et al., 2024) are gradual architectural adjustments that accelerate training and reduce inference cost. In computer vision, progressive model growing has been explored for image generation (Karras et al., 2017) and segmentation (Fischer et al., 2024).

In particular, progressive stacking is an approach that trains a model in stages, adding new layers (randomly initialized or copied) at the start of each stage until a final model size is reached. Despite the intuitive appeal of progressive stacking as an approach to learn hierarchical features and improve compute efficiency, recent studies suggest it underperforms in knowledge-intensive tasks such as closed-book question answering. For example, MIDAS (Saunshi et al., 2024) finds that progressively grown models lag behind full-capacity models trained from scratch on memorization-focused tasks. In our own setup and compute-controlled experiments, we also find that progressive growing performs similarly or worse than standard training on several benchmarks.

We hypothesize that model expansion alone is insufficient

¹Stanford University. Correspondence to: Karanpartap Singh <karanps@stanford.edu>.

for compute-efficient learning of hierarchical features, and investigate whether stacking should be paired with a structured learning signal to realize its potential. Inspired by cognitive development, we propose **Curriculum-Guided Layer Scaling (CGLS)**, a pretraining paradigm that couples progressive model expansion with a data curriculum of gradually increasing complexity. To preserve and build on previously learned representations during model growth, CGLS uses staged training: at each step, newly added transformer layers are first trained in isolation (with earlier layers frozen), before the entire model is fine-tuned on a more complex data distribution. This strategy protects previously learned representations during capacity increases and aligns model complexity with data difficulty throughout training.

Our Contributions. In summary, we make the following contributions:

- We propose **Curriculum-Guided Layer Scaling (CGLS)**, a unified framework for jointly scaling model capacity and data complexity via progressive layer expansion and curriculum learning.
- We conduct a rigorous empirical evaluation of CGLS at two parameter counts and compute scales, demonstrating its scalability and robustness across both general pretraining and domain-shift settings.
 - At a GPT-2-Small parameter count using a synthetic-to-webtext curriculum, CGLS achieves consistent gains in perplexity and downstream QA performance (1.0% on average, and +3.0% on ARC-Easy).
 - At a Llama-3.2-1B parameter count and 2.5B tokens, we stratify the DataComp-LM (DCLM) corpus (Li et al., 2024) using a DistilBERT-based classifier (Sanh et al., 2019) and show that CGLS outperforms compute-matched baselines on knowledge-intensive and reasoning-heavy tasks, on average by 1.7% across benchmarks and by as much as 5.0% on ARC-Easy. Scaling up to a Chinchilla-optimal 20B tokens (Hoffmann et al., 2022), gains are even larger, with an average 3.90% increase across benchmarks.

Overall, we demonstrate that joint scaling of model and data complexity unlocks the benefits of progressive stacking for language model pretraining, yielding better downstream performance on knowledge-intensive tasks up to the 1B parameter and 20B tokens scale.

Conflict of Interest Disclosure. The authors declare no financial conflicts of interest related to this work.

2. Related Work

Curriculum learning, formalized by Bengio et al. (2009), has shown empirical benefits in many domains, from vi-

sion (Hacohen & Weinshall, 2019; Weinshall et al., 2018) to reinforcement learning (Graves et al., 2017). However, it remains underexplored in the context of large language model (LLM) pretraining, where most models are trained in a single pass over a diverse and unstructured dataset. Self-paced learning strategies (Kumar et al., 2010) adjust training dynamics based on model uncertainty, gradually introducing harder samples as the model becomes more confident. More recently, efforts such as DataComp (Li et al., 2024), MetaCLIP (Xu et al., 2023), and Tower (Alves et al., 2024) demonstrate that large performance gains can arise from careful dataset curation, even without architectural changes. Related work like Thrush et al. (2024) and Ankner et al. (2024) investigates the use of small language models to identify high-quality data subsets that better align with LLM performance, but does not explore staged training. Likewise, Gururangan et al. (2020) show the value of domain-specific continued pretraining after generic training, indicating that structured data exposure plays a key role even in later stages. A closely related curriculum-learning baseline is Beyond Random Sampling (BRS) (Zhang et al., 2026), which evaluates curricula with ten difficulty groups and finds that fine-grained groupings provide little advantage over a simpler 3-tier curriculum. Still, these approaches do not consider whether data curricula coordinated with model growing could improve compute efficiency.

Parallel to this, a growing body of research examines how to progressively increase model capacity during training. Soviany et al. (2021) and others have noted that the curriculum principle may apply not just to data, but also to model size. In vision, progressive training has been used to stabilize training and reduce compute costs, e.g., by gradually growing generative networks (Karras et al., 2017) or using function-preserving transformations (Chen et al., 2015). For Transformers, techniques such as LayerDrop (Fan et al., 2020) and Width-Adaptive Transformers (Zhao et al., 2024) enable dynamic capacity at inference time, though they are not designed for staged training. Mixture-of-Experts (MoE) models (Shazeer et al., 2017; Fedus et al., 2022) similarly allocate compute conditionally across tokens but often suffer from expert collapse or shallow specialization to token-level features (Zoph et al., 2022). Alongside the vast progressive stacking literature mentioned in Section 1, more recently, architectural methods have emerged to progressively grow models during training. Wang et al. (2023) propose learning an explicit mapping from a smaller transformer’s parameters into a larger one, enabling weight reuse and transfer. Complementary to this, Yao et al. (2023) introduce smoother transitions between training stages by gradually increasing the influence of new parameters. Du et al. (2024) compare the various stacking strategies in prior work, and demonstrate their scalability and optimal usage. Such efforts underscore the promise of synchronized curricula for data and model:

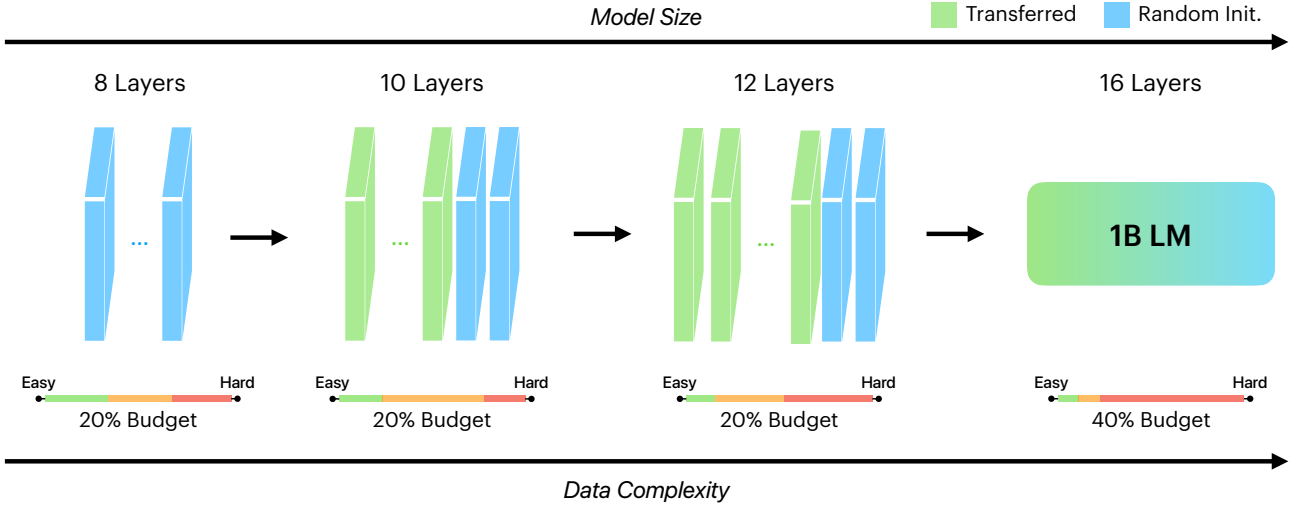


Figure 1. **Curriculum-Guided Layer Scaling (CGLS)** is a new paradigm for compute-efficient language model pretraining that grows data complexity and model depth in tandem. We illustrate CGLS for a Llama-3.2-1B scale model with four training stages. Training begins with an 8-layer model on a data split consisting equally of data from all levels (high-school, undergraduate, and graduate). The learned weights from this stage are transferred to a larger 10-layer model, freezing the pretrained weights and training the new layers on a small, balanced data split for better initialization. The entire model is then unfrozen and pretrained on the more difficult data split. This process is repeated until the target model scale is reached.

gradually increasing data complexity or domain diversity in tandem with growing model capacity or specialization. Our Curriculum-Guided Layer Scaling approach unifies prior ideas of curriculum learning and progressive model scaling into a single coordinated pretraining strategy.

3. Method

Humans do not learn language by memorizing the full dictionary on day one. Instead, we begin with simple patterns: repetitive sounds, basic grammar, and simple visual inputs, and gradually expand to handle more abstract concepts, longer sentences, and complex reasoning as our cognitive capacities mature. This progression in data complexity is tightly coupled with the brain’s developmental growth: as new neural structures form, they scaffold increasingly sophisticated representations of the world (Kolk & Rakic, 2022). We study whether this notion of growing neural and data complexity in tandem can improve the compute efficiency of language model pretraining.

Previous work has explored increasing either neural or data complexity alone during language model pretraining, through curriculum learning and various approaches for expanding neural nets; we review these approaches in Section 2 above. Inspired by the structured development of the human brain, we develop and study a pretraining paradigm that synchronizes a data curriculum with the gradual addition of randomly initialized layers to the language model. We next describe these two components in our framework—

curriculum and layer expansion—in turn.

Curriculum Learning. To construct a pretraining curriculum, we organize documents into progressively harder splits, beginning with simpler and more structured samples before introducing more complex, diverse, and noisier data. Let $\mathcal{D}_i$ represent the dataset used at stage i , where $|\mathcal{D}_i|$ is the number of tokens. We first segment our pretraining dataset into easy, medium, and hard components $\mathcal{D}_{\text{Easy}}$, $\mathcal{D}_{\text{Medium}}$ and $\mathcal{D}_{\text{Hard}}$. Then, the data distribution for stage i can be expressed as a mixture of the easy, medium, and hard components with mixture weights (p_i, q_i, r_i) , $p_i + q_i + r_i = 1$:

$$\mathcal{D}_i = p_i \cdot \mathcal{D}_{\text{Easy}} + q_i \cdot \mathcal{D}_{\text{Medium}} + r_i \cdot \mathcal{D}_{\text{Hard}}.$$

The mixture weights (p_i, q_i, r_i) are adjusted to balance each stage’s difficulty, as shown in Figure 1.

Layer Expansion. In parallel with a structured curriculum, we expand the model’s capacity at each stage by incrementally adding transformer layers. Beginning with a shallow model (e.g., 6 layers), we train on simpler datasets before transferring the learned weights to a larger model with additional layers. Let $\Theta^{(i)}$ denote the parameters of the model at stage i , and $\Theta_{1:k}^{(i)}$ the parameters of the first k non-embedding layers. Then, at stage $i + 1$, the parameters are initialized as:

$$\Theta^{(i+1)} := \{\Theta_{\text{embed}}^{(i)} \circ \Theta_{1:k}^{(i)} \circ \Theta_{k+1:n}^{\text{random}} \circ \Theta_{\text{norm}}^{(i)} \circ \Theta_{\text{lm}}^{(i)}\}$$

where $\Theta_{k+1:n}^{\text{random}}$ represents the newly added, randomly initialized layers, and $\circ$ denotes concatenation of parameter groups corresponding to the different components of the model. Following the results of Kumar et al. (2022), training is then performed in two phases:

1. **Initialization phase:** the new parameters $\Theta_{k+1:n}^{\text{random}}$ are trained, while all other layers are frozen; that is, the embedding layer $\Theta_{\text{embed}}^{(i)}$, previous non-embedding layers $\Theta_{1:k}^{(i)}$, normalization layer $\Theta_{\text{norm}}^{(i)}$, and language modeling head $\Theta_{\text{lm}}^{(i)}$ are all kept frozen.
2. **Full tuning phase:** all parameters in $\Theta^{(i+1)}$ are optimized.

This approach aims to ensure that the representations learned in prior layers are effectively utilized and not overwritten when the model size is increased. This progression is illustrated in Figure 1. In practice, we find that CGLS performs well with a simple default configuration across model scales: initializing with half the final depth ($N_1 = N/2$), allocating more compute to later stages (e.g., 20/20/20/40%), and dedicating 20% of each stage to new-layer initialization. We use this configuration throughout most experiments.

4. Experiments

The core intuition of our approach for compute-efficient pretraining is to follow a slow transition from highly structured to more diverse data while scaling model capacity. We construct pretraining curricula using the TinyStories (Eldan & Li, 2023), BookCorpus (Zhu et al., 2015), and DataComp for Language Models (DCLM) (Li et al., 2024) corpora. We first implement our approach at a GPT-2 Small scale (Radford et al., 2019), starting with a small model trained on simple, synthetic language patterns (TinyStories) and gradually increasing both data complexity and model size. Then, we scale our approach to the parameter count of Llama-3.2-1B (Dubey et al., 2024), applying it to a stratified version of the DCLM dataset, where document complexity levels (e.g., high school, undergraduate, graduate) are inferred via a DistilBERT classifier trained on a small set of GPT-4o-labeled samples. Next, we describe this particular instantiation of data curricula that we used in our experiments.

4.1. Data Curricula

In conventional LLM pretraining, noisy and diverse datasets (e.g., Common Crawl or The Pile) are used early, and high-quality curated datasets are reserved for the final stages (Hu et al., 2024), or fine-tuning (Ouyang et al., 2022). This approach assumes that early exposure to broad linguistic patterns enhances generalization, while later fine-tuning reduces noise artifacts (Brown et al., 2020). In contrast, we explore whether incorporating more structure in early

stages and later transitioning to diverse corpora can improve downstream performance. We find that different curricula work best at our two different scales and describe them in depth next.

GPT-2-Small Scale Curriculum. For the GPT-2 experiments, we start with a split containing samples primarily from TinyStories, consisting of synthetically-generated short stories comprehensible to a young child. The goal of training on such highly-structured data is to establish robust low-level linguistic representations. We then gradually introduce more challenging datasets, such as BookCorpus and DCLM, characterized by broader vocabulary, complex sentence structures, and diverse contexts. Our hypothesis is that early exposure to clean, consistent patterns helps the model develop reliable embeddings, which later support generalization to noisier data such as DCLM.

Llama-3.2-1B Scale Curriculum. At the 1B scale, rather than using separate datasets of increasing complexity, we stratified the DCLM-Baseline-1.0 corpus itself into tiers of general, domain-specific, and specialized reasoning complexity. This enabled us to perform curriculum progression entirely within a single large corpus, while leveraging the model’s increased capacity to handle more complexity earlier in training as compared to the small-scale experiments. To implement this, we first perform *complexity stratification* using a separate pretrained LLM:

1. **Constructing Classifier Training Set:** Using GPT-4o-mini, we label a subset of 20,000 documents by classifying complexity into three levels:
 - *High School:* Structured text with limited abstraction or formal reasoning, such as news articles, beginner-level Wikipedia pages, or simple conversational stories.
 - *Undergraduate:* Content with moderate reading difficulty and some domain-specific concepts, including ArXiv abstracts and science articles that require basic abstract reasoning.
 - *Graduate/Advanced:* Highly technical or specialized content such as academic papers in AI/ML, legal documents, medical research studies, and code repositories (e.g., GitHub).
2. **Class Balancing & Splitting:** We then upsampled any underrepresented categories, and performed a random 80/10/10 split for training, validation, and testing.
3. **Model-Based Classification:** Finally, we trained a simple DistilBERT-based classifier (Sanh et al., 2019) on this data, which achieved $> 90\%$ test-set accuracy and was used to label the full 2M document subset of DCLM used for training. Then, by choosing mixture weights over the stratified splits of DCLM, our data schedule (see

Appendix A.1) reflects a gradual shift from general to specialized content as model depth increases.

4.2. Compute-Matched Method Comparisons

In our experiments at the GPT-2-Small and Llama-3.2-1B parameter counts, we fix a compute budget in total pretraining FLOPs across all methods. Since CGLS trains shallower models in early stages, this fixed FLOP budget corresponds to processing more token updates during those stages than a full-depth model would process for the same compute. Thus, all methods use the same data and final architecture, but the progressive methods allocate the fixed compute budget across models of increasing depth. When early-stage data is exhausted, we continue sampling from the corresponding data mixture, so CGLS may revisit examples, but does not receive additional unique data.

Prior work on LLM scaling suggests that optimal training lies near the “Chinchilla” compute/data tradeoff curve (Hoffmann et al., 2022), i.e., with $20\times$ more tokens than parameters. In our initial experiments, we use the FLOP budget corresponding to 2.5B full-depth training tokens for both the GPT-2-Small and Llama-3.2-1B scale experiments. Then, we perform a focused larger-scale investigation of our method at the Chinchilla-optimal ratio of 1B parameters and 20B tokens, in addition to a 4B-token domain-shift experiment transitioning from general text to code. Following well-established practices in the LLM scaling laws literature (Kaplan et al., 2020), we approximate the total compute budget for each experiment, in FLOPs, as:

$$\text{Total FLOPs} = 6 \cdot \text{Tokens} \cdot \text{Parameters} \cdot \text{Epochs}$$

Our experiments consisted of training several compute-matched models under our CGLS framework as well as a set of natural baseline approaches applying curricula over data and model complexity.

CGLS Configuration. Training begins with $N/2$ layers, where N is the desired final model depth, trained on a curriculum-balanced dataset (see detailed data splits in Appendix A.1), followed by a staged progression to the full model size. At each stage, newly added layers are first trained with a fixed data split (used for all initialization phases), before the full model is fine-tuned on a progressively more challenging data distribution.

Compute-Matched Baselines. For both model scales, we evaluate the following compute-matched baselines under the same pretraining FLOP budget:

- **Randomized baseline:** A model with the final depth (a “full model”) is trained from scratch on a randomized mixture of the available data.

- **Curricularized baseline:** This baseline simulates a coarse curriculum by training the full model sequentially on increasingly complex data subsets. For GPT-2-Small, we train on TinyStories, then BookCorpus, then DCLM. For Llama-3.2-1B, we follow the same staged training but use the stratified DCLM splits (high school, undergraduate, graduate). Both this curricularized baseline and CGLS use the same difficulty classifier, the same three difficulty tiers, and the same pacing schedule; all optimization hyperparameters and compute budgets are matched. The only difference is whether the model depth is expanded during training.
- **Layer-scaling only:** This baseline incrementally expands the model depth across stages, in the same schedule and manner as CGLS, but trains on the full (unstructured) data distribution at each stage without a curriculum. This isolates the contribution of progressive stacking alone.
- **MIDAS:** MIDAS progressively stacks layers across training stages but does not control the complexity of the training data. We reimplemented the method from pseudocode provided in the original paper (Saunshi et al., 2024), as there is no public codebase, with PROP-3 stage budgets, a block size of 3 for GPT-2-Small, and a block size of 4 for Llama-3.2-1B. Layer weights from previous stages were copied at each model expansion, as in the original paper, and FLOPs were matched with the other baselines and CGLS.

Further CGLS Implementation Details. At GPT-2-Small parameter scale, we scaled from 6 to 12 layers in 3 stages; at Llama-3.2-1B parameter scale, from 8 to 16 layers in 4 stages. As explored in the Appendix A.3, initializing with half of the final model depth, and allocating more training budget to the later stages (e.g. 20% each for the first three stages; 40% for the final stage) consistently yielded strong performance and serves as a reliable default. Further details on our experimental setup are provided in the Appendix A.1.

4.3. Evaluation

We evaluate the generalization and reasoning capabilities of the models on several downstream tasks:

- **PIQA (Physical Interaction QA)** (Bisk et al., 2019): This dataset tests physical commonsense reasoning using multiple-choice questions about everyday scenarios.
- **ARC Easy and Challenge** (Clark et al., 2018): The AI2 Reasoning Challenge benchmarks measure a model’s ability to answer elementary and middle-school science questions. The “Easy” subset contains relatively straightforward questions, while the “Challenge” subset consists of harder, multi-step reasoning tasks.
- **HellaSwag** (Zellers et al., 2019): This task evaluates

commonsense reasoning and sentence completion, requiring the model to identify the most plausible continuation for a given context. Given our constrained academic compute budget, raw accuracy on multiple-choice tasks tends to have low signal-to-noise (Heineman et al., 2025). Instead, we evaluate *Token-Normalized Accuracy* (Kydliček et al., 2024; Gu et al., 2024), a metric that better accounts for length and bias effects in generative modeling to obtain a low-variance signal at small scales. This is defined as:

$$\text{acc}_{\text{token}} = \arg \max_i \frac{\ln P(a_i | q)}{\text{num_tokens}(a_i)}$$

- **LAMBADA** (Paperno et al., 2016): An open-book language modeling task designed to assess a model’s ability to perform long-range dependency reasoning. We report zero-shot accuracy, computed as whether the model correctly predicts the final word of a passage.

At our largest experiment scale of 1B parameters trained on 20B tokens, we additionally test the following benchmarks:

- **GPQA (Graduate-Level Physics QA)** (Rein et al., 2023): A challenging benchmark designed to assess high-level scientific and mathematical reasoning. Questions are drawn from graduate physics coursework and require multi-step conceptual understanding.
- **TLDR9+ (Summarization)** (Sotudeh et al., 2021): A summarization task based on Reddit posts, requiring abstractive generation of concise summaries from long, noisy inputs. We report Rouge-L (Lin, 2004) following prior work.
- **OpenRewriteEval (ORE)** (Shu et al., 2023): An open-ended rewriting and paraphrasing benchmark. The model must rewrite user-provided text to satisfy semantic constraints such as preserving meaning, altering tone, or improving clarity. We report Rouge-L (Lin, 2004) here as well.
- **InfiniteBench EN.QA (Long-Context QA)** (Zhang et al., 2024): A long-context benchmark where models must answer questions requiring retrieval and reasoning over very lengthy contexts. We report the English QA subset (EN.QA), which stresses information retention and retrieval at sequence lengths far beyond typical context windows. We report ROUGE F1 scores as specified for this benchmark.

All tasks are evaluated in a zero-shot setting to measure base model quality, following recent academic pretraining works such as (Hwang et al., 2026), although we include few-shot results in A.2. We report the token-normalized accuracy, as described above, for all multiple-choice tasks. We additionally compute perplexity on a 180,000 sample subset of the Pile dataset, capturing how well the methods model diverse, noisy data from a variety of domains.

5. Results

Our experiments evaluate the performance of CGLS at two parameter counts (matched to GPT-2-Small and Llama-3.2-1B) across perplexity and zero-shot accuracy on several downstream tasks. Table 1 summarizes the findings, comparing CGLS to several baselines with matched FLOP budgets. Across both model scales, GPT-2-Small and Llama-3.2-1B, CGLS consistently outperforms baseline approaches on reasoning-focused benchmarks such as ARC and PIQA, demonstrating its effectiveness in improving generalization under fixed compute budgets. CGLS shows particular strength in structured and knowledge-intensive tasks, supporting the benefits of aligning model growth with data complexity.

GPT-2-Small Scale. The GPT-2 scale CGLS model achieves consistent improvements on downstream tasks, as illustrated in Table 1. For example, CGLS attains a 43.01% accuracy on ARC-Easy, a 4.1-point improvement over the randomized baseline, along with gains on PIQA and Hel-laSwag. While improvements are generally modest at this scale, all methods are strongly capacity-limited and these experiments are therefore presented as a controlled proof-of-concept setting.

Llama-3.2-1B Scale. The larger models demonstrate clearer trends. While the randomized baseline achieves the lowest perplexity across all Pile domains (see A.9 for detailed perplexity results), CGLS achieves the strongest downstream performance. Across nearly all downstream tasks, CGLS improves over the best baseline: e.g., on PIQA (61.21% vs. 59.09%), ARC-Easy (41.37% vs. 36.36%). The gains on ARC-Easy are especially striking, exceeding the best baseline by over 5 percent. **Notably, the stacking baselines, without a synchronized data curricula, did not improve downstream performance.**

5.1. Extending CGLS

Scaling to a 1B Chinchilla-Optimal Token Budget. To test whether the benefits of CGLS persist at larger training scales, we replicate our main CGLS pretraining experiment on a 1B parameter model trained on 20B tokens, matching the Chinchilla-optimal parameter-to-data ratio (Hoffmann et al., 2022). At this scale, we observe in Table 3 that CGLS delivers clear gains over the randomized baseline across all benchmarks, and in perplexity on The Pile. Improvements are particularly notable on PIQA (+4.4 points), ARC-Easy (+5.2 points), and LAMBADA (+3.7 points), with the average accuracy rising from 38.4% to 42.3%. These results demonstrate that curriculum-guided layer scaling continues to confer benefits even as both model and dataset sizes grow, suggesting that its inductive bias remains useful at more realistic pretraining scales.

Curriculum-Guided Layer Scaling for Language Model Pretraining

Model	The Pile [↓]	PIQA [↑]	ARC-E [↑]	ARC-C [↑]	HellaSwag [↑]	LAMBADA [↑]	Average [↑]
GPT-2-Small							
Pretrained GPT-2-Small*	23.97	62.51%	39.48%	22.70%	31.14%	32.56%	38.96%
Baseline (Randomized)	<u>27.93</u>	60.94%	38.85%	<u>23.38%</u>	26.89%	25.58%	35.13%
Baseline (Curricularized)	26.89	60.61%	40.03%	22.53%	<u>27.17%</u>	26.12%	35.29%
Layer Scaling Only	32.10	60.23%	38.43%	21.93%	27.05%	27.05%	34.94%
MIDAS (Saunshi et al., 2024)	30.70	60.77%	37.58%	24.40%	26.77%	28.29%	<u>35.56%</u>
CGLS	29.31	61.64%	43.01%	23.12%	27.21%	<u>28.08%</u>	36.61%
Llama-3.2-1B							
Pretrained Llama-3.2-1B*	9.25	74.32%	60.56%	36.52%	63.67%	62.00%	59.41%
Baseline (Randomized)	20.83	<u>59.09%</u>	<u>36.36%</u>	<u>24.24%</u>	<u>34.20%</u>	<u>32.99%</u>	<u>37.38%</u>
Baseline (Curricularized)	<u>21.93</u>	58.27%	35.19%	25.43%	34.65%	33.09%	37.33%
Layer Scaling Only	29.53	57.45%	32.58%	23.46%	30.00%	27.73%	34.24%
MIDAS (Saunshi et al., 2024)	27.64	55.44%	32.58%	24.23%	29.62%	25.87%	33.55%
CGLS	25.34	61.21%	41.37%	25.43%	34.65%	32.74%	39.08%

Table 1. Evaluation results comparing CGLS and baseline approaches across average perplexity on The Pile dataset and core reasoning- and knowledge-intensive downstream tasks. CGLS improves accuracy on downstream tasks like PIQA compared to the baselines. All models are compute-matched, but utilize much less compute than the pretrained variants of these models. *Not compute-matched to the other methods, and trained on orders-of-magnitude more data. We provide pretrained results to illustrate a rough oracle upper bound for the given architecture. The best method among the compute-matched approaches is bolded, with the second-best method underlined.

Model	pass@8	pass@16	pass@32	pass@64
Llama-3.2-1B				
Baseline (Rand.)	3.56%	4.26%	5.27%	6.71%
CGLS	6.67%	7.63%	8.78%	9.76%

Table 2. Results for a domain-shift experiment from general-purpose web documents (DCLM) to code data (StarCoder2), using HumanEval for downstream evaluation. As a baseline, we trained a 1B parameter model on a randomized 50/50 mix of DCLM and StarCoder2 totaling 4B tokens. CGLS was trained on precisely the same data in a different order: a curriculum gradually transitioning from general text to code, and achieves higher pass@k scores for the full range of k considered.

Domain-Adaptive Code Pretraining. Beyond scaling within a single corpus, we evaluate whether the coordinated data and model curriculum of CGLS can improve compute-efficiency in a domain-adaptive pretraining setting (Gururangan et al., 2020). Specifically, we instantiate this setting by evaluating CGLS under domain shift from general-purpose web text (2B tokens from DCLM) to code data (2B tokens of Python code from StarCoder2). Using HumanEval for downstream evaluation (Table 2), the baseline model trained on the randomized mixture of DCLM and StarCoder2 achieves modest pass@k performance.

In contrast, CGLS—trained on the same 4B tokens but ordered to progressively transition from general text to code—consistently outperforms the baseline in pass@k, e.g., nearly doubling pass@8 (6.7% vs. 3.6%). These re-

sults highlight that CGLS not only improves performance under fixed compute budgets, but also provides a structured mechanism for navigating distribution shifts between domains. More details on this experiment, including the data curriculum split, are provided in Appendix A.1.

5.2. Additional Analyses

Beyond overall benchmark comparisons, we investigate two complementary aspects of CGLS. First, we assess stability through repeated training runs at the 2.5B-token scale (Section A.4), and through replacing our DistilBERT-based difficulty classifier with the FineWeb educational classifier (Penedo et al., 2024) (Section A.5). Across three random seeds, CGLS achieves higher performance than the baseline across all benchmarks, indicating that the method’s gains are robust to stochasticity in optimization. When using the FineWeb classifier, CGLS attains an average accuracy nearly identical to that obtained with the DistilBERT classifier, suggesting that the method is robust to the specific curriculum signal used to stratify data.

Second, we analyze the average validation accuracy across curriculum stages (Section A.10). These curves provide insight into how different training phases contribute to downstream performance. We observe steady gains for CGLS, particularly in later stages, whereas baseline methods plateau earlier. This progression underscores the role of synchronized model and data scaling in sustaining learning

Curriculum-Guided Layer Scaling for Language Model Pretraining

Model	Pile [↓]	PIQA [↑]	ARC-E [↑]	ARC-C [↑]	HS [↑]	LBD [↑]	Avg [↑]	GPQA [↑]	T9+ [↑]	ORE [↑]	IB [↑]
Llama-3.2-1B (20B Tokens)											
BL-R	26.33	63.00%	41.79%	23.38%	33.52%	30.37%	38.41%	<u>25.22%</u>	2.787	<u>17.30%</u>	0.235%
BL-C	<u>24.58</u>	<u>63.55%</u>	<u>42.97%</u>	<u>24.83%</u>	<u>34.61%</u>	<u>32.66%</u>	<u>39.72%</u>	22.32%	<u>2.947</u>	20.61%	<u>0.401%</u>
CGLS	22.99	67.36%	46.97%	26.28%	36.80%	34.08%	42.30%	29.24%	3.568	17.03%	0.539%

Table 3. Evaluation results comparing CGLS and baseline approaches (BL-R refers to the compute-matched baseline trained on randomized data, and BL-C refers to the compute-matched baseline trained with a curriculum) at the 20B-token scale, aligned with the 20× data/parameter ratio in (Hoffmann et al., 2022). All models are compute-matched. The listed benchmarks are: perplexity on The Pile (Pile), PIQA (PIQA), ARC-Easy (ARC-E), ARC-Challenge (ARC-C), HellaSwag (HS), LAMBADA (LBD), the average over MC-QA tasks (Avg), GPQA (GPQA), TLDR9+ summarization (T9+), OpenRewriteEval (ORE), and the InfiniteBench long-context EN.QA task (IB). The best score for each metric is bolded, with the second-best method underlined.

dynamics throughout training. Additional hyperparameter and stage length ablations are provided in Section A.3.

6. Discussion

Inspired by how humans acquire cognitive capabilities in tandem with brain maturation and exposure to progressively more complex information, we align model capacity with data complexity during language model pretraining. We propose a new pretraining paradigm, Curriculum-Guided Layer Scaling (CGLS), that grows models in depth while adapting the data distribution to match their evolving capabilities. Applied to pretraining at GPT-2-Small and Llama-3.2-1B scales, our approach yields consistent improvements in downstream performance, with this trend holding at larger 1B-Chinchilla training scales and for domain-adaptive code pretraining. In compute-matched comparisons, CGLS models outperform baselines on benchmarks requiring structured reasoning and domain-specific understanding, such as ARC and PIQA. Importantly, stacking-based baselines that expand model depth without synchronized data curricula do not improve downstream performance—CGLS significantly outperforms them across key benchmarks, demonstrating that aligning model growth with data complexity can unlock the benefits of progressive growing. Altogether, our results establish CGLS as a promising new paradigm for compute-efficient language model pretraining.

Limitations. While CGLS yields improvements across several knowledge-intensive and reasoning tasks, in a few tasks such as LAMBADA and HellaSwag, CGLS models perform comparably to the baselines, suggesting that not all tasks benefit equally from the progressive curriculum. We also observe that CGLS does not always lower perplexity: at both the GPT-2-Small and Llama-3.2-1B scales with 2.5B tokens, the randomized baseline achieved slightly better perplexity despite underperforming on reasoning tasks. Our intuition for this result is provided by studies of scaling laws with respect to downstream task performance (Tay et al., 2022a;b), which find that some model inductive biases such as depth matter disproportionately more for downstream

tasks than pretraining perplexity. CGLS is a curriculum over layer depth and therefore we postulate that there may be a related mechanism at play. This disconnect between perplexity and downstream performance highlights the need for better evaluation frameworks.

Future Work. Our experiments suggest that the initial model depth is a particularly important hyperparameter: starting with half the final number of layers ($N_1 = N/2$) yielded the strongest performance at both pretraining scales. Starting too small (e.g., $N_1 = 6$ at Llama-3.2-1B scale) led to weaker final models, likely due to insufficient capacity to capture basic linguistic patterns, while starting too large (e.g., $N_1 = 10$) provided no benefit and in some cases underperformed standard training (see Section A.3). This points to a sweet spot in progressive scaling where initial tractability and long-term transferability are best balanced. Additionally, our experiments with the FineWeb classifier suggest that curriculum construction is an important design choice, and that exploring improved or domain-adaptive difficulty classifiers may further strengthen CGLS. Broadly, these results highlight a rich design space for curriculum-guided scaling. Future work could explore (1) more finely tuning stage lengths and dataset proportions, (2) exploring other notions—e.g., syntactic diversity, lexical richness, or factuality—for defining the curriculum, (3) applying curriculum-aware optimization schedules (e.g., varying learning rates with data complexity), (4) extending CGLS to larger parameter regimes (e.g. 7B), and (5) applying CGLS to non-text modalities (e.g., vision or multimodal data).

Acknowledgments. This work was supported by the Stanford Institute for Human-Centered AI (HAI) Hoffman-Yee Award, UST, and National Institutes of Health (NIH) Grant AG089169. KS was supported by the National Science Foundation (NSF) Graduate Research Fellowship Program under Grant No. 2023331708. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the NSF or NIH.

Impact Statement

This work aims to improve the compute efficiency of language model pretraining by studying how model capacity and data complexity can be jointly scheduled during training. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here. We do not release new datasets containing sensitive information, and our experiments use standard public corpora and evaluation benchmarks.

References

- Abdin, M., Aneja, J., Awadalla, H., Awadallah, A., and others. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. 4 2024.
- Alves, D. M., Pombal, J., Guerreiro, N. M., Martins, P. H., Alves, J., Farajian, A., Peters, B., Rei, R., Fernandes, P., Agrawal, S., Colombo, P., de Souza, J. G. C., and Martins, A. F. T. Tower: An Open Multilingual Large Language Model for Translation-Related Tasks. 2 2024.
- Ankner, Z., Blakeney, C., Sreenivasan, K., Marion, M., Leavitt, M. L., and Paul, M. Perplexed by Perplexity: Perplexity-Based Data Pruning With Small Reference Models. 5 2024.
- Bengio, Y., Louradour, J., Collobert, R., and Weston, J. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pp. 41–48, New York, NY, USA, 6 2009. ACM. ISBN 9781605585161. doi: 10.1145/1553374.1553380.
- Bisk, Y., Zellers, R., Bras, R. L., Gao, J., and Choi, Y. PIQA: Reasoning about Physical Commonsense in Natural Language. 11 2019.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language Models are Few-Shot Learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M. F., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- Chen, T., Goodfellow, I., and Shlens, J. Net2Net: Accelerating Learning via Knowledge Transfer. 11 2015.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. 3 2018.
- Du, W., Luo, T., Qiu, Z., Huang, Z., Shen, Y., Cheng, R., Guo, Y., and Fu, J. Stacking Your Transformers: A Closer Look at Model Growth for Efficient LLM Pre-Training. 10 2024.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., and others. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*, 7 2024.
- Eldan, R. and Li, Y. TinyStories: How Small Can Language Models Be and Still Speak Coherent English?, 2023. URL <https://arxiv.org/abs/2305.07759>.
- Fan, A., Grave, E., and Joulin, A. Reducing Transformer Depth on Demand with Structured Dropout. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=Syl02yStDr>.
- Fan, S. and Jaggi, M. Irreducible Curriculum for Language Model Pretraining. 10 2023.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.*, 23(1), 1 2022. ISSN 1532-4435.
- Feng, S., Prabhumoye, S., Kong, K., Su, D., Patwary, M., Shoenybi, M., and Catanzaro, B. Maximize Your Data’s Potential: Enhancing LLM Accuracy with Two-Phase Pretraining. 12 2024.
- Fischer, S. M., Felsner, L., Osuala, R., Kiechle, J., and others. Progressive Growing of Patch Size: Resource-Efficient Curriculum Learning for Dense Prediction Tasks. 7 2024.
- Gong, L., He, D., Li, Z., Qin, T., Wang, L., and Liu, T.-Y. Efficient Training of BERT by Progressively Stacking. In *2019 International Conference on Machine Learning*, pp. 2337–2346. PMLR, 5 2019. URL <https://proceedings.mlr.press/v97/gong19a.html>.
- Graves, A., Bellemare, M. G., Menick, J., Munos, R., and Kavukcuoglu, K. Automated Curriculum Learning for Neural Networks. 4 2017.
- Gu, X., Liu, L., Yu, H., Li, J., Chen, C., and Han, J. On the Transformer Growth for Progressive BERT Training. 10 2020.

- Gu, Y., Tafjord, O., Kuehl, B., Haddad, D., Dodge, J., and Hajishirzi, H. OLMES: A Standard for Language Model Evaluations. 6 2024.
- Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., and Smith, N. A. Don’t Stop Pre-training: Adapt Language Models to Domains and Tasks. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 8342–8360, Online, 7 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.740. URL <https://aclanthology.org/2020.acl-main.740/>.
- Hacohen, G. and Weinshall, D. On The Power of Curriculum Learning in Training Deep Networks. 4 2019.
- Heineman, D., Hofmann, V., Magnusson, I., Gu, Y., Smith, N. A., Hajishirzi, H., Lo, K., and Dodge, J. Signal and Noise: A Framework for Reducing Uncertainty in Language Model Evaluation. 8 2025.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. d. L., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., Driessche, G. v. d., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L. Training Compute-Optimal Large Language Models. 3 2022.
- Hu, S., Tu, Y., Han, X., He, C., and others. MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies. 4 2024.
- Hwang, S., Wang, B., and Gu, A. Dynamic Chunking for End-to-End Hierarchical Sequence Modeling. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=ZbfLR9NbNF>.
- J. Reddi, S., Miryoosefi, S., Karp, S., Krishnan, S., and others. Efficient Training of Language Models using Few-Shot Learning. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 14553–14568. PMLR, 5 2023. URL <https://proceedings.mlr.press/v202/j-reddi23a.html>.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., and others. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*, 1 2020.
- Karras, T., Aila, T., Laine, S., and Lehtinen, J. Progressive Growing of GANs for Improved Quality, Stability, and Variation. 10 2017.
- Kim, B.-K., Kim, G., Kim, T.-H., Castells, T., and others. Shortened LLaMA: Depth Pruning for Large Language Models with Comparison of Retraining Methods. 2 2024.
- Kolk, S. M. and Rakic, P. Development of prefrontal cortex. *Neuropsychopharmacology*, 47(1):41–57, 1 2022. ISSN 0893-133X. doi: 10.1038/s41386-021-01137-9.
- Kumar, A., Raghunathan, A., Jones, R., Ma, T., and Liang, P. Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution. 2 2022.
- Kumar, M., Packer, B., and Koller, D. Self-Paced Learning for Latent Variable Models. In Lafferty, J., Williams, C., Shawe-Taylor, J., Zemel, R., and Culotta, A. (eds.), *Advances in Neural Information Processing Systems*, volume 23. Curran Associates, Inc., 2010. URL https://proceedings.neurips.cc/paper_files/paper/2010/file/e57c6b956a6521b28495f2886ca0977a-Paper.pdf.
- Kydlíček, H., Penedo, G., Fourier, C., Habib, N., and others. FineTasks: Finding signal in a haystack of 200+ multilingual tasks, 2024. URL <https://huggingface.co/spaces/HuggingFaceFW/blogpost-fine-tasks>.
- Li, J., Fang, A., Smyrnis, G., Ivgi, M., Jordan, M., Gadre, S., Bansal, H., Guha, E., Keh, S., Arora, K., Garg, S., Xin, R., Muennighoff, N., Heckel, R., Mercat, J., Chen, M., Gururangan, S., Wortsman, M., Albalak, A., Bitton, Y., Nezhurina, M., Abbas, A., Hsieh, C.-Y., Ghosh, D., Gardner, J., Kilian, M., Zhang, H., Shao, R., Pratt, S., Sanyal, S., Ilharco, G., Daras, G., Marathe, K., Gokaslan, A., Zhang, J., Chandu, K., Nguyen, T., Vasiljevic, I., Kakade, S., Song, S., Sanghavi, S., Faghri, F., Oh, S., Zettlemoyer, L., Lo, K., El-Nouby, A., Pouransari, H., Toshev, A., Wang, S., Groeneveld, D., Soldaini, L., Koh, P. W., Jitsev, J., Kollar, T., Dimakis, A. G., Carmon, Y., Dave, A., Schmidt, L., and Shankar, V. DataComp-LM: In search of the next generation of training sets for language models. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 14200–14282. Curran Associates, Inc., 2024. doi: 10.52202/079017-0455.
- Lin, C.-Y. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*, pp. 74–81, Barcelona, Spain, 7 2004. Association for Computational Linguistics. URL <https://aclanthology.org/W04-1013/>.
- OpenAI, Achiam, J., Adler, S., Agarwal, S., and others. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 3 2023.

- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback. 3 2022.
- Panigrahi, A., Saunshi, N., Lyu, K., Miryoosefi, S., Reddi, S., Kale, S., and Kumar, S. Efficient Stagewise Pretraining via Progressive Subnetworks. 2 2024.
- Paperno, D., Kruszewski, G., Lazaridou, A., Pham, Q. N., Bernardi, R., Pezzelle, S., Baroni, M., Boleda, G., and Fernández, R. The LAMBADA dataset: Word prediction requiring a broad discourse context. 6 2016.
- Penedo, G., Kydlíček, H., allal, L. B., Lozhkov, A., Mitchell, M., Raffel, C., Von Werra, L., and Wolf, T. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. 10 2024.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language Models are Unsupervised Multi-task Learners. 2019.
- Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. 11 2023.
- Sanh, V., Debut, L., Chaumond, J., and Wolf, T. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. 10 2019.
- Saunshi, N., Karp, S., Krishnan, S., Miryoosefi, S., and others. On the Inductive Bias of Stacking Towards Improving Reasoning. 9 2024.
- Schnabel, T., Tomlinson, K., Swaminathan, A., and Neville, J. Lost in Transmission: When and Why LLMs Fail to Reason Globally. 5 2025.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. 1 2017.
- Shu, L., Luo, L., Hoskore, J., Zhu, Y., Liu, Y., Tong, S., Chen, J., and Meng, L. RewriteLM: An Instruction-Tuned Large Language Model for Text Rewriting. 12 2023.
- Sotudeh, S., Deilamsalehy, H., Dernoncourt, F., and Goharian, N. TLDR9+: A Large Scale Resource for Extreme Summarization of Social Media Posts. 10 2021.
- Soviany, P., Ionescu, R. T., Rota, P., and Sebe, N. Curriculum Learning: A Survey. 1 2021.
- Tay, Y., Dehghani, M., Abnar, S., Chung, H. W., Fedus, W., Rao, J., Narang, S., Tran, V. Q., Yogatama, D., and Metzler, D. Scaling Laws vs Model Architectures: How does Inductive Bias Influence Scaling? 7 2022a.
- Tay, Y., Dehghani, M., Rao, J., Fedus, W., Abnar, S., Chung, H. W., Narang, S., Yogatama, D., Vaswani, A., and Metzler, D. Scale Efficiently: Insights from Pretraining and Finetuning Transformers. In *International Conference on Learning Representations*, 2022b. URL <https://openreview.net/forum?id=f2OYVDyfiB>.
- Thrush, T., Potts, C., and Hashimoto, T. Improving Pretraining Data Using Perplexity Correlations. 9 2024.
- Wang, P., Panda, R., Hennigen, L. T., Greengard, P., Karlinsky, L., Feris, R., Cox, D. D., Wang, Z., and Kim, Y. Learning to Grow Pretrained Models for Efficient Transformer Training. 3 2023.
- Weinshall, D., Cohen, G., and Amir, D. Curriculum Learning by Transfer Learning: Theory and Experiments with Deep Networks. 2 2018.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., and others. HuggingFace’s Transformers: State-of-the-art Natural Language Processing. 10 2019.
- Xu, H., Xie, S., Tan, X. E., Huang, P.-Y., Howes, R., Sharma, V., Li, S.-W., Ghosh, G., Zettlemoyer, L., and Feichtenhofer, C. Demystifying CLIP Data. 9 2023.
- Yang, C., Wang, S., Yang, C., Li, Y., He, R., and Zhang, J. Progressively Stacking 2.0: A Multi-stage Layerwise Training Method for BERT Training Speedup. 11 2020.
- Yao, Y., Zhang, Z., Li, J., and Wang, Y. Masked Structural Growth for 2x Faster Language Model Pre-training. 5 2023.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. HellaSwag: Can a Machine Really Finish Your Sentence? 5 2019.
- Zhang, M. and He, Y. Accelerating Training of Transformer-Based Language Models with Progressive Layer Dropping. 10 2020.
- Zhang, X., Chen, Y., Hu, S., Xu, Z., Chen, J., Hao, M. K., Han, X., Thai, Z. L., Wang, S., Liu, Z., and Sun, M. ∞ Bench: Extending Long Context Evaluation Beyond 100K Tokens. 2 2024.
- Zhang, Y., Mohamed, A., Abdine, H., Shang, G., and Vazirgiannis, M. Beyond Random Sampling: Efficient Language Model Pretraining via Curriculum Learning. In Demberg, V., Inui, K., and Marquez, L. (eds.), *Proceedings of the 19th Conference of the European Chapter*

of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 5776–5794, Rabat, Morocco, 3 2026. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.271. URL <https://aclanthology.org/2026.eacl-long.271/>.

Zhao, W., Han, Y., Tang, J., Wang, K., Song, Y., Huang, G., Wang, F., and You, Y. Dynamic Diffusion Transformer. 10 2024.

Zhu, Y., Kiros, R., Zemel, R., Salakhutdinov, R., and others. Aligning Books and Movies: Towards Story-like Visual Explanations by Watching Movies and Reading Books. 6 2015.

Zoph, B., Bello, I., Kumar, S., Du, N., Huang, Y., Dean, J., Shazeer, N. M., and Fedus, W. ST-MoE: Designing Stable and Transferable Sparse Expert Models. 2022. URL <https://api.semanticscholar.org/CorpusID:248496391>.

A. Appendix

GPT-2-Small (124M Parameters)	
Stage 1	70 / 10 / 20
Stage 2	15 / 25 / 60
Stage 3	5 / 5 / 90
New Layer Tuning	45 / 30 / 25

Table 4. Data curricula per training stage for the GPT-2-Small parameter scale setup. Each split is represented as a proportion of TinyStories, BookCorpus, and DCLM tokens with format TinyStories / BookCorpus / DCLM, used for tuning only the new layers (New Layer Tuning) or for full tuning of the entire model at that stage.

Llama-3.2-1B (1.2B Parameters)			
Stage	General (HS)	Domain (UG)	Specialized (Grad.)
Stage 1	25%	50%	25%
Stage 2	20%	40%	40%
Stage 3	15%	35%	50%
Stage 4	10%	10%	80%
New Layer Tuning	34%	33%	33%

Table 5. Data curricula per training stage for the Llama-3.2-1B parameter scale setups. The same set of optimized mixture weights are used for all CGLS experiments. Each stage is defined by mixture weights over three levels of complexity: high school-level (general understanding), undergraduate-level (some domain-specific knowledge), or graduate-level (highly specialized information).

A.1. Experimental Setup

All training was performed using the Hugging Face Transformers library (Wolf et al., 2019) on NVIDIA H100 80GB GPUs with NVLink. Each model was initialized with a learning rate of 2×10^{-4} and optimized using AdamW with a weight decay of 0.01. A warmup-stable-decay (WSD) learning rate schedule was applied, with 1,000 warmup steps during the initial stage (smallest model), followed by a stable learning rate across intermediate stages, and cosine decay to zero during the final stage (largest model). Layers added after model expansion were initially trained with a higher learning rate of 5×10^{-4} , for 20% of the compute budget for the given stage. Optimizer states were reset between stages for all experiments involving layer expansion.

The full stage-wise splits of TinyStories, BookCorpus, and DCLM for the GPT-2-Small experiments are provided in Table 4. For Llama-3.2-1B, the stratified mixture weights over general, domain-specific, and specialized subsets of DCLM are in Table 5. For the domain shift experiment from DCLM (general web text) to StarCoder2 (Python code), we used a four-stage progression with mixture weights of 80/20, 70/30, 60/40, and 50/50 (DCLM/StarCoder), with all other hyperparameters identical to the other experiments.

Training the full-depth 1B-parameter baseline for 2.5B tokens took 5 h 39 min on four H100 GPUs, while the corresponding FLOP-matched CGLS configuration took 5 h 53 min. We note that our implementation of CGLS is not yet optimized for efficiency, e.g., with checkpoint loading from a fast disk. Moreover, no aspects of CGLS should meaningfully decrease model FLOPs utilization (MFUs), and CGLS and the baseline are compute-matched. Given this, we anticipate the wall-clock time of CGLS and the baseline should be essentially identical.

A.2. Few Shot Results

In standard academic pretraining works, few-shot evaluations are uncommon because effective in-context learning emerges at larger scales, as shown in the GPT-3 paper (Brown et al., 2020). For example, recent large-scale academic works such as H-Nets (Hwang et al., 2026), which pretrains 1B parameter models for 100B tokens, do not include few-shot evaluations. Nevertheless, we conduct some few-shot evaluations, with PMI accuracy (Gu et al., 2024) to reduce variance, and report the results in Table 6. For both zero- and 5-shot evaluation, CGLS achieves the best average across the benchmarks.

Model	# Shots	PIQA [↑]	ARC-E [↑]	ARC-C [↑]	HellaSwag [↑]	LAMBADA	Average [↑]
Llama-3.2-1B w/ 2.5B Tokens							
Baseline (Randomized)	0	54.41%	32.69%	23.98%	31.41%	29.89%	34.48%
	5	56.31%	33.15%	26.19%	31.32%	31.32%	35.66%
Baseline (Curricularized)	0	53.86%	27.30%	30.19%	30.19%	34.02%	35.11%
	5	54.73%	29.27%	29.73%	29.73%	33.57%	35.41%
Layer-Scaling Only	0	53.54%	26.71%	28.89%	28.46%	27.67%	33.05%
	5	54.95%	26.71%	29.38%	29.11%	28.37%	33.70%
MIDAS	0	51.58%	26.54%	30.19%	28.89%	25.91%	32.62%
	5	57.18%	28.33%	31.81%	29.38%	29.42%	35.22%
CGLS	0	56.69%	33.75%	29.18%	31.95%	31.04%	36.52%
	5	58.65%	33.65%	29.10%	32.02%	31.95%	37.07%

Table 6. Zero- and 5-shot PMI-normalized evaluation results across benchmarks. CGLS achieves the best average at both shot settings (bold). Note: because this table uses PMI-normalized scoring, the zero-shot values are not directly comparable to the token-normalized accuracies in Table 1.

A.3. Hyperparameter Ablations

We conduct ablations of core hyperparameters such as stage-wise training budgets and the starting depth. Though we are not able to exhaustively tune these hyperparameters on our academic compute budget, we report below some results conducting 1D sweeps on these key parameters. We find in Figure 2 that starting at half the final depth ($N_1 = 8$) consistently yields the best results across downstream tasks, and that allocating 40% of the compute budget to the final stage performs best, outperforming front-heavy (25%) and back-heavy (50%, 75%) schedules. Increasing the starting depth beyond half the final depth did not yield additional improvements. These findings suggest that a balanced schedule that gradually expands capacity optimizes performance. Additionally, starting from half the model depth may strike a near-optimal balance between early-stage tractability and representational richness in progressive training frameworks.

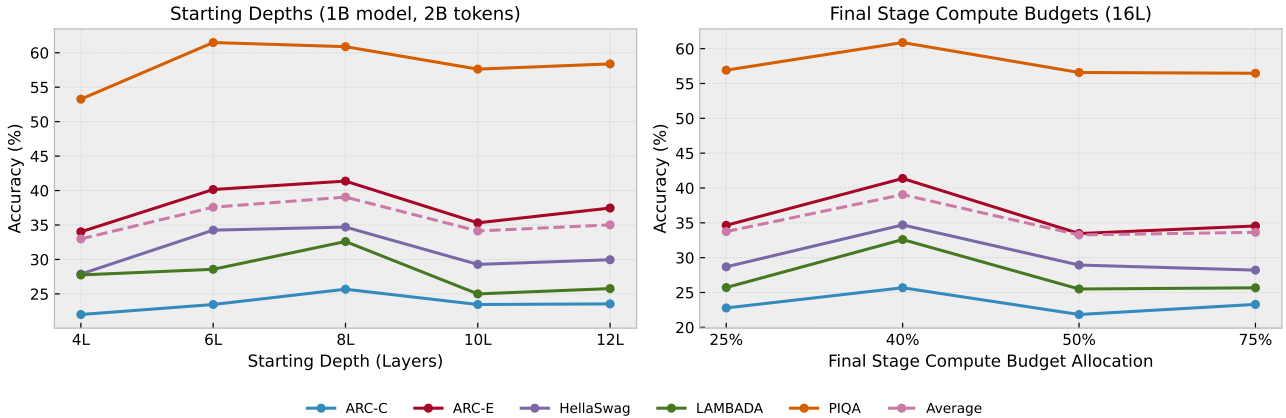


Figure 2. Results for 1D hyperparameter sweeps over the starting model depth and stage-wise training budgets for the 1B-parameter model, trained on 2.5B tokens from DCLM. The average across the benchmarks is denoted with a dashed line.

A.4. Stability

Replication experiments at the 1B scale (Table 7) confirm that the improvements from CGLS are consistent across multiple seeds. While the variance across runs is slightly larger for CGLS than for the baseline, the mean accuracy is higher on every benchmark, indicating that the method’s benefits are robust. This suggests that progressive scaling may introduce some additional variability in optimization dynamics, but reliably yields stronger final models.

Curriculum-Guided Layer Scaling for Language Model Pretraining

Model	PIQA [†]	ARC-E [†]	ARC-C [†]	HellaSwag [†]	LAMBADA [†]	Average [†]
Llama-3.2-1B w/ 2.5B Tokens						
Baseline	59.18 $\pm$ 0.28%	36.46 $\pm$ 0.42%	22.38 $\pm$ 0.78%	27.81 $\pm$ 0.24%	20.49 $\pm$ 0.36%	33.26 $\pm$ 0.24%
CGLS	62.19 $\pm$ 1.42%	40.47 $\pm$ 2.43%	23.78 $\pm$ 1.76%	30.06 $\pm$ 1.24%	23.22 $\pm$ 1.19%	35.94 $\pm$ 1.60%

Table 7. Experimental replication results comparing CGLS and baseline approaches at the 2.5B token scale. Both models are compute-matched with training runs repeated three times in each case. We report zero-shot results with standard errors, with CGLS achieving a higher score across all benchmarks.

Model	PIQA [†]	ARC-E [†]	ARC-C [†]	HellaSwag [†]	LAMBADA [†]	Average [†]
Llama-3.2-1B w/ 2.5B Tokens						
Baseline	59.09	36.36	24.24	34.20	32.99	37.38
CGLS	61.21	41.37	25.43	34.65	32.74	39.08
FineWeb	61.59	45.20	26.37	31.30	30.84	39.06

Table 8. Comparison of CGLS using the default DistilBERT-based difficulty classifier vs. a FineWeb-based difficulty classifier. All benchmarks are conducted in a zero-shot setting. FineWeb yields similar average performance to standard CGLS, with stronger results on ARC-E and ARC-C but lower performance on HellaSwag and LAMBADA.

A.5. Robustness to the Difficulty Classifier

A core component of CGLS is the curriculum used to order training data by difficulty. In our primary experiments, difficulty stratification is performed using a DistilBERT-based classifier trained to predict coarse complexity labels. Since model performance may depend on the quality or structure of this classifier, we assess the robustness of CGLS to alternate curriculum signals.

To this end, we conducted an additional experiment in which we replaced the DistilBERT-based classifier with the FineWeb educational difficulty classifier (Penedo et al., 2024), which provides a substantially different distributional prior and difficulty spectrum. Table 8 reports results for the baseline, the original CGLS configuration, and the variant that uses FineWeb for curriculum construction.

Across benchmarks, the FineWeb-based CGLS variant performs comparably to the DistilBERT-based version: both substantially outperform the baseline, and their average accuracies (39.06% vs. 39.08%) are nearly identical. The FineWeb classifier produces modest improvements on PIQA, ARC-Easy, and ARC-Challenge, while slightly reducing performance on HellaSwag and LAMBADA. These shifts reflect differences in how the two classifiers stratify training data, but the overall stability indicates that CGLS is not overly sensitive to the specific classifier used to define difficulty.

These findings suggest that CGLS benefits from the existence of a meaningful progression in data complexity, rather than from the idiosyncrasies of a particular classifier. We therefore view improved, domain-adaptive, or multi-signal difficulty estimators as an important direction for future work, and emphasize that CGLS is compatible with a wide family of curriculum-generation strategies beyond DistilBERT.

A.6. Preservation of Early Representations

To evaluate whether CGLS better preserves early learned representations, we measure perplexity of the final models on data from both early and late curriculum stages. CGLS consistently achieves lower perplexity than the baseline on both the earliest-stage mixtures and the subsets containing the simplest examples, indicating stronger retention of previously learned distributions. These results suggest that synchronizing model growth with data complexity helps stabilize learned representations and mitigate forgetting during later stages of training.

A.7. Freezing Phase Ablation

To better understand the role of the freezing/initialization phase in CGLS, we conduct an ablation where newly added layers are introduced and jointly optimized immediately with the existing network, without first freezing the earlier layers. All other components of the training setup remain unchanged.

Eval Subset	Baseline PPL $\downarrow$	CGLS PPL $\downarrow$	Δ (Baseline - CGLS)	Relative Gain
Llama-3.2-1B w/ 2.5B Tokens				
Stage 1 Mixture	26.9636	24.4975	2.4661	9.15%
Simplest Examples	27.9900	25.6431	2.3468	8.38%
Stage 4 Mixture	28.6047	25.7281	2.8766	10.06%
Hardest Examples	28.8622	25.8518	3.0103	10.43%

Table 9. Retention analysis comparing perplexity of the final baseline and CGLS models on early- and late-stage curriculum subsets. CGLS consistently achieves lower perplexity, suggesting improved preservation of earlier learned representations and reduced forgetting during progressive training.

The results are shown in Table 10. Removing the freezing phase leads to mixed behavior: performance improves slightly on some benchmarks (e.g., PIQA and ARC-E), but degrades substantially on others, particularly HellaSwag and LAMBADA, resulting in worse average performance overall. We interpret this as evidence of a stability–plasticity tradeoff. Without freezing, newly introduced layers adapt more rapidly, which can benefit certain tasks, but this also increases interference with previously learned representations.

To further evaluate this effect, we measure perplexity on the first stage’s training mixture after the completion of training. The baseline, CGLS, and no-freeze variants achieve perplexities of 26.96, 24.50, and 30.84 respectively, indicating substantially worse retention of early-stage representations without the freezing phase. These results suggest that the initialization phase is an important component of CGLS for stabilizing optimization and preserving representations learned during earlier curriculum stages.

Model	PIQA $\uparrow$	ARC-E $\uparrow$	ARC-C $\uparrow$	HellaSwag $\uparrow$	LAMBADA $\uparrow$	Average $\uparrow$
Llama-3.2-1B w/ 2.5B Tokens						
Baseline	59.09	36.36	24.24	34.20	32.99	37.38
CGLS	61.21	41.37	25.43	34.65	32.74	39.08
CGLS (No Freeze)	62.02	42.21	24.57	31.28	27.21	37.46

Table 10. Ablation evaluating the role of the freezing/initialization phase in CGLS. Removing freezing improves performance on some benchmarks (e.g., PIQA and ARC-E), but substantially hurts performance on others, particularly HellaSwag and LAMBADA, leading to lower overall average accuracy. These results suggest that the freezing phase improves training stability and helps preserve previously learned representations during progressive layer expansion.

A.8. Curriculum Desynchronization

To further isolate the role of synchronization between model growth and data complexity, we conduct a *desynchronization* experiment in which we reverse the curriculum order while keeping all other factors fixed, including the staged layer-growth procedure, compute budget, and datasets. If the gains from CGLS were primarily due to generic aspects of the training recipe (e.g., staged optimization or freezing/thawing), we would expect performance to remain comparable. Instead, reversing the curriculum substantially degrades performance across all benchmarks, falling below both the baseline and the original CGLS configuration. While reversing the curriculum order is a substantial intervention, these results, together with the ablations presented in the main body, suggest that the benefits of CGLS arise specifically from synchronizing model growth with progressively increasing data complexity.

A.9. Perplexity Results

For both GPT-2-Small and Llama-3.2-1B trained on 2.5B tokens, we observe that CGLS yields higher perplexity than the baselines across most subsets of The Pile (Figures 3,4). As discussed in the main paper, this divergence highlights that perplexity does not always align with downstream task improvements, particularly at smaller training budgets. Notably, scaling to a Chinchilla-optimal 20B tokens, this discrepancy disappears, and CGLS improves both perplexity and downstream accuracy.

Model	PIQA [†]	ARC-E [†]	ARC-C [†]	HellaSwag [†]	LAMBADA [†]	Average [†]
Llama-3.2-1B w/ 2.5B Tokens						
Baseline	59.09	36.36	24.24	34.20	32.99	37.38
CGLS	61.21	41.37	25.43	34.65	32.74	39.08
CGLS (Desync)	56.64	32.41	23.46	27.97	22.12	32.52

Table 11. Desynchronization experiment where the curriculum order is reversed while keeping all other aspects of CGLS fixed. Reversing the curriculum substantially degrades performance across all benchmarks, suggesting that synchronization between model growth and increasing data complexity is critical to the effectiveness of CGLS.

Perplexity Across The Pile — GPT-2-Small

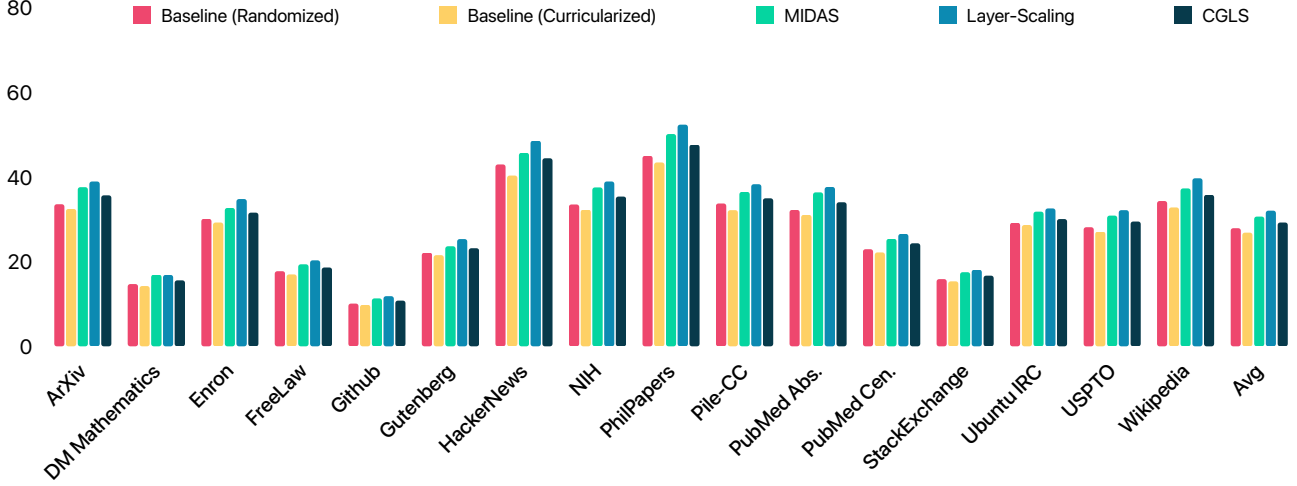


Figure 3. Model perplexity at GPT-2-Small parameter count across various individual subsets of The Pile, for CGLS and all baselines. Lower perplexity scores indicate better performance. Across all subsets, the curricularized baseline achieves the lowest perplexity, with progressive scaling models outperforming MIDAS and layer scaling without a data curriculum.

Perplexity Across The Pile — Llama 3.2 (1B)

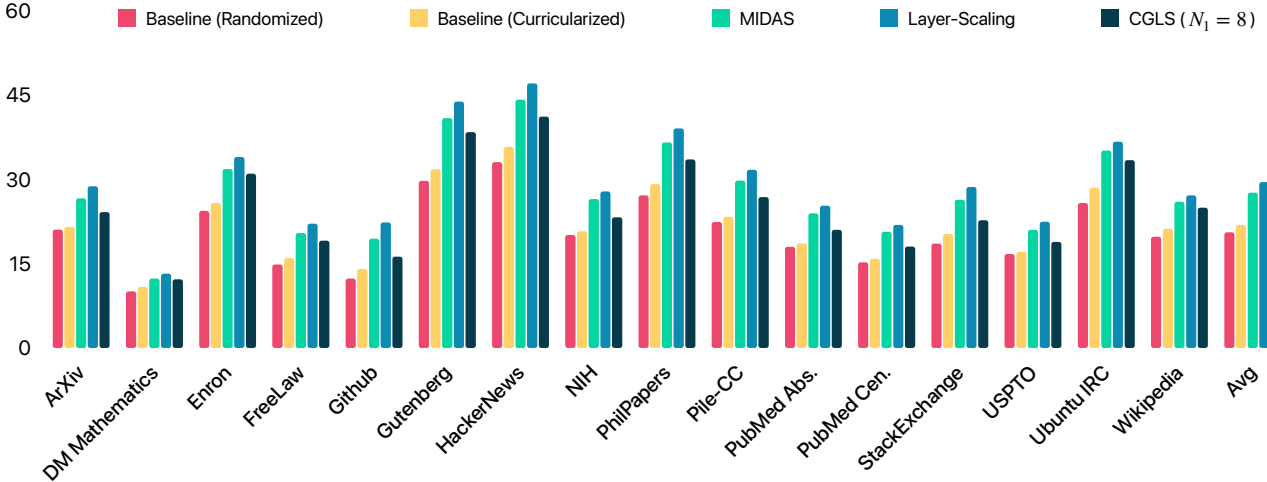


Figure 4. Model perplexity across various individual subsets of The Pile for all baselines versus CGLS for Llama-3.2-1B. Lower perplexity scores indicate better performance. Across all subsets, the randomized baseline achieves the lowest perplexity, with progressive scaling models outperforming MIDAS and layer scaling without a data curriculum.

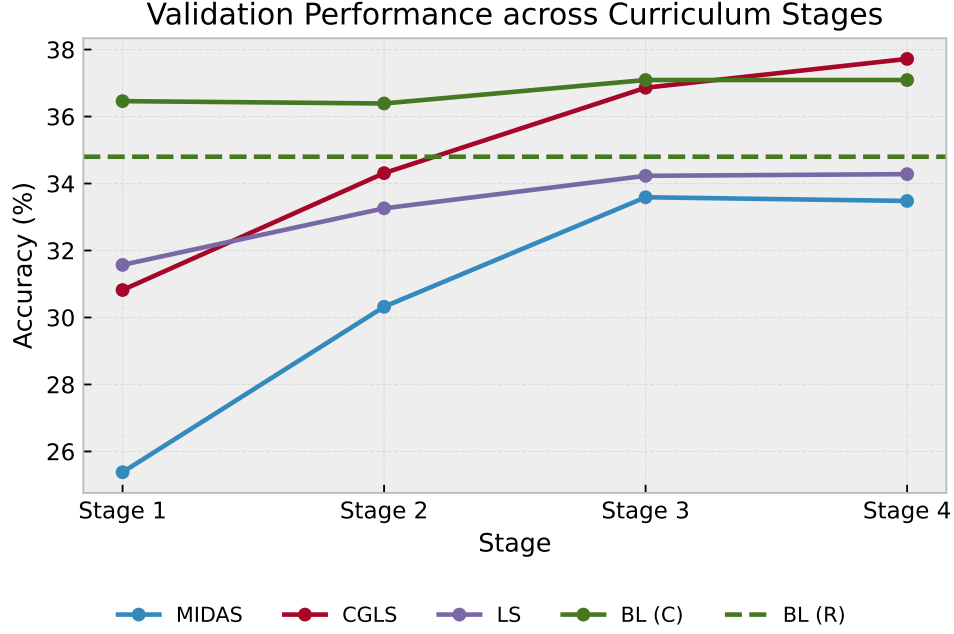


Figure 5. Validation performance at each training stage for all methods. BL refers to the baseline, with R denoting the randomized variant, and C the curricularized variant.

A.10. Performance by Stage

Figure 5 reports the average accuracy across the benchmarks after each stage of training for CGLS and competing methods. The results highlight two main trends. First, CGLS improves steadily across stages, surpassing the randomized baseline by Stage 3 and continuing to gain performance in the final expansion. Second, while alternative progressive strategies such as MIDAS and LS also show early improvements, their performance plateaus sooner, converging below the baseline reference. In contrast, CGLS maintains upward momentum throughout the curriculum, suggesting that its synchronized data and model growth more effectively transfers knowledge from earlier stages into the final model.